

The Warm Session

A project file that is a value; a daemon that is the project as a process; a CLI that is nothing but a remote control for it.

begun 2026-09-11 · this revision 2026-09-12

Two mechanisms for running a project's code already existed — a warm daemon and an isolated test runner — and they did not know about each other. This report follows the work that makes them one thing: **the warm session**. Each wave below states what changed, the decision behind it, and the evidence that it works, taken from the session in which it was built.

What this is

beam-lisp's `bl` CLI grew organically: twenty-one verbs, each its own namespace, each learning how to find a project's files on its own. Alongside them, a daemon learned how to keep a VM warm for a tree, and a test runner (`ward`) learned how to run a file in an isolated fork. Warmth and isolation were **commands**.

This work makes them **properties of execution**. The surface that results:

<code>bl daemon start</code>	the one lifecycle noun — a warm session for this tree
<code>env.bl</code>	the project as a value: roots, tasks, ports, environment
<code>bl <task></code>	a task the project declares, running against the warm image
<code>bl test</code>	isolated per file, always — <code>ward</code> 's semantics, no second runner
<code>bl ui</code>	the session's dashboard: the same read-model the terminal shows
<code>bl mcp</code>	the same capabilities, for editors and agents

The laws this work establishes

LAW · ONE MEANING, TWO HOSTS

A command behaves identically cold (a standalone VM) and warm (through the daemon). Warmth is an optimization, never a difference in meaning: both hosts call the same entry point, and every wave is verified against both.

LAW · ERRORS ARE VALUES

A wrong key in a project file, a malformed file, a tree with no project file — each degrades what a tree can do and is reported as data. None of them stops a command, and none of them stops the daemon.

LAW · ONE IMPLEMENTATION PER NEED

Nothing in this work exists twice. The project file has one reader; the test semantics have one runner reachable cold and warm; the dashboard is a pure view over one read-model, and the terminal is another view of the same one.

How to read the evidence

Three kinds of block carry the argument:

- **decision** — a choice and the reason for it. If you disagree with a choice, the reason is what to attack.
- **proof** — something observed in this session, with the command that observed it. The transcripts are real output, not illustrations.
- **dogfooded** — how the new tooling was used to check the new tooling.

Anything not yet true is in a **gap** block, marked as such.

The waves

Each wave ships on its own and is verified before the next begins.

W1	The project file — <code>env.bl</code> read cold and warm	shipped
W2	Tasks — <code>bl <name></code> as a declaration	shipped
W3	The session's address — named ports, one URL, MCP on it	shipped
W4	<code>bl test</code> absorbs <code>ward</code>	planned
W5	One read-model, two faces	shipped
W6	<code>bl ui</code> — the dashboard, and MCP over HTTP	shipped
W7	Walkthroughs that run, and a repo that reads its own file	shipped

W1 The project file — env.bl read cold and warm

SHIPPED, 13 TESTS

A tree can now describe itself in one file at its root. The file is beam-lisp, its last form is a map, and every command reads it before it does anything:

```
{:name "pulse"
 :paths ["src" "priv"]           ; library roots — replaces -p flags
 :tasks {:dev {:run "src/main.bl" :watch true :doc "the dev loop"}
         :seed "scripts/seed.bl"}
 :ports {:web 4000 :metrics {:port 0}} ; named; 0 = the OS chooses
 :env   {"PULSE_ENV" "dev"}}
```

The three rules that make it trustworthy

DECISION

Discovery walks **up** from the command's own directory.

Because — `bl` typed three levels down finds the same file a command at the root finds, so no command ever needs a path to its own configuration — `direnv`'s rule, and the reason the file belongs to a tree rather than a directory.

DECISION

The file has two readers that cannot disagree: a **data** read (parse the last form, evaluate nothing) and a **fork** read (evaluate the file in its own env).

Because — Tooling that must not run project code — the doctor, the language server, an editor — still sees exactly what a run sees. A file whose value is computed reports `:dynamic` from the cheap read, and the caller falls back to the fork.

DECISION

Every shape problem is collected into `:errors` as data, and the project map is always the right shape.

Because — A configuration bug that silently does nothing is the worst kind. A file with six mistakes reports six, and the command that read it still runs — with the paths, tasks and ports it could understand.

```
$ bl eval '(... (bl.env/project "..."))' — a deliberately broken env.bl
```

```
{:name nil, :paths [], :tasks {}, :ports {}, :env {},
 :errors ("...:paths must be a list of strings"
         "...:name must be a string"
         "task \"a\" must be a file name or a map"
         "port \"w\" must be a number or {:port N}"
         "...:env must be a map of name → string"
         "unknown key wat")}}
```

six problems, one pass, nothing raised, no valid key discarded

What a command binds

`bl.cli` reads the project once per command, before dispatch, and binds two things: the declared roots onto the loader's search path, and the declared environment into the process. Cold and warm both come through this one door.

✓ A library resolves from a declared root, with no flags

v(0.35em) block(width: 100%, inset: (left: 1.1em))[A scratch tree with `env.bl` declaring `:paths ["src"]`, a namespace at `src/lib/thing.bl`, and a program requiring it. Run from a directory three levels below the root:

```
$ mix bl run ../src/app.bl (from <tree>/sub/dir)
```

```
hi from lib, mode=engaged
```

the root came from env.bl; the mode came from :env — neither from a flag

]

✓ The same tree through the daemon

v(0.35em) block(width: 100%, inset: (left: 1.1em))[`test/beam_lisp/daemon_env_test.exs` boots a real daemon for this repository and sends protocol requests whose CLIENTS sit in a scratch tree — including from a subdirectory. Both assertions are about the client's project, not the daemon's checkout:

```
$ mix test test/beam_lisp/daemon_env_test.exs
```

```
02 tests, 0 failures · the daemon bound the CLIENT's env.bl
```

]

The bug this wave uncovered

The daemon binds a request's roots through `Loader.with_ambient_dirs/2` — the client's cwd plus its `-p` paths — because the daemon VM's own cwd is the checkout, not the client's tree. That binding **replaced** the standalone capture `[File.cwd!() | extra_dirs()]`, and `extra_dirs()` is where a session's configured search paths live. A search path added inside a daemon request — which is exactly what a project's `:paths` are — was therefore invisible to the load: `add_search_path` writes into the request's `fork`, while the `Loader.Server` that loads namespaces reads its own env's paths.

DECISION

The capture is now additive: the caller's bound roots **plus** its configured search paths, in either mode.

Because — One expression, both hosts. The alternative — teaching the daemon's Elixir side to compute project roots itself — would have put the same logic in two places and let the two hosts drift, which is precisely what the whole design forbids.

Instead of — making the Elixir executor read `env.bl` and append paths to `ambient_dirs/1` (a second implementation of “which roots apply”).

LAW · THE FORK IS NOT A SOUND BARRIER

Configuration a request binds for itself must reach the component that acts on it, even when that component runs in another process on a different env. A search path is session configuration, not process state.

Numbers, and how they were checked

NEW NAMESPACE

`priv/std/bl/env.bl.md` — `bl.env` (~250 lines, livebook)

find · literal · evaluate · normalize · project

TESTS

11 tests · 33 assertions (`test/bl/system/env_test.bl`)

discovery at depth · both reads · fork isolation · shape errors · normalization · cold binding · a broken file that still runs

WARM-PATH TEST

2 ExUnit cases (`test/beam_lisp/daemon_env_test.exs`)

a real daemon, real protocol frames, the client's tree

NEIGHBOURING SUITES

38 ExUnit (daemon) · 334 bl tests (system)

re-run after the loader change; 0 failures

DOGFODEED

Every test in this wave was run through the warm runner under construction — `bl test ...` — not only through `mix test`, and the daemon cases drive the same `run-argv` entry a `bl` invocation uses. The next wave wires the daemon's own task and port surface; the report will show those running through the session they describe.

W3 The session's address — named ports, one URL, and MCP on it

SHIPPED

A project declares the ports it serves on, the session claims them by name, and the session itself answers on one of them:

```
{:ports {:web 4000 :metrics {:port 0} :ui 7700}}
```

`bl daemon start` prints what it became:

```
$ bl daemon start
```

```
bl daemon up for /home/user/code/undefine/beam-lisp
```

```
ui: http://127.0.0.1:44779 (ephemeral — pin it in env.bl with :ports {:ui 7700})
```

```
mcp: http://127.0.0.1:44779/mcp (the same MCP `bl mcp` serves over stdio)
```

the port is ephemeral unless the project pins it, and the message says exactly how to pin it

DECISION

The registry is FILES — one per claim, under the runtime dir — not a table in the daemon.

Because — The collision worth catching is between *trees*: two checkouts, two daemons, one port 4000.

A table inside one VM cannot see the other. A file can, it survives a crash (which is exactly when a stale claim needs sweeping), and it carries the owner — so the refusal can say **whose** port it is:

```
$ a second session claims a port the first holds
```

```
{:error, {:taken, %{name: "web", port: 4000, root: "/code/pulse", pid: 90210}}}
```

an error that names the tree, the pid and the port — not ``address already in use``

Instead of — a `GenServer` mirroring `WatchRegistry` (the plan's original sketch), which would have been one more in-VM table that cannot answer the cross-daemon question.

LAW · A PORT NOBODY CHOSE IS NOBODY'S

`{:port 0}` asks the OS and records what it chose, so two trees on one machine never fight over a number. A pinned number that is taken is REFUSED, never quietly replaced by another — a URL that silently moves is worse than a startup that fails.

One URL for the whole session

The `:ui` port serves the session page **and** the MCP endpoint. An editor, an agent and a browser need one address, not a registry of them.

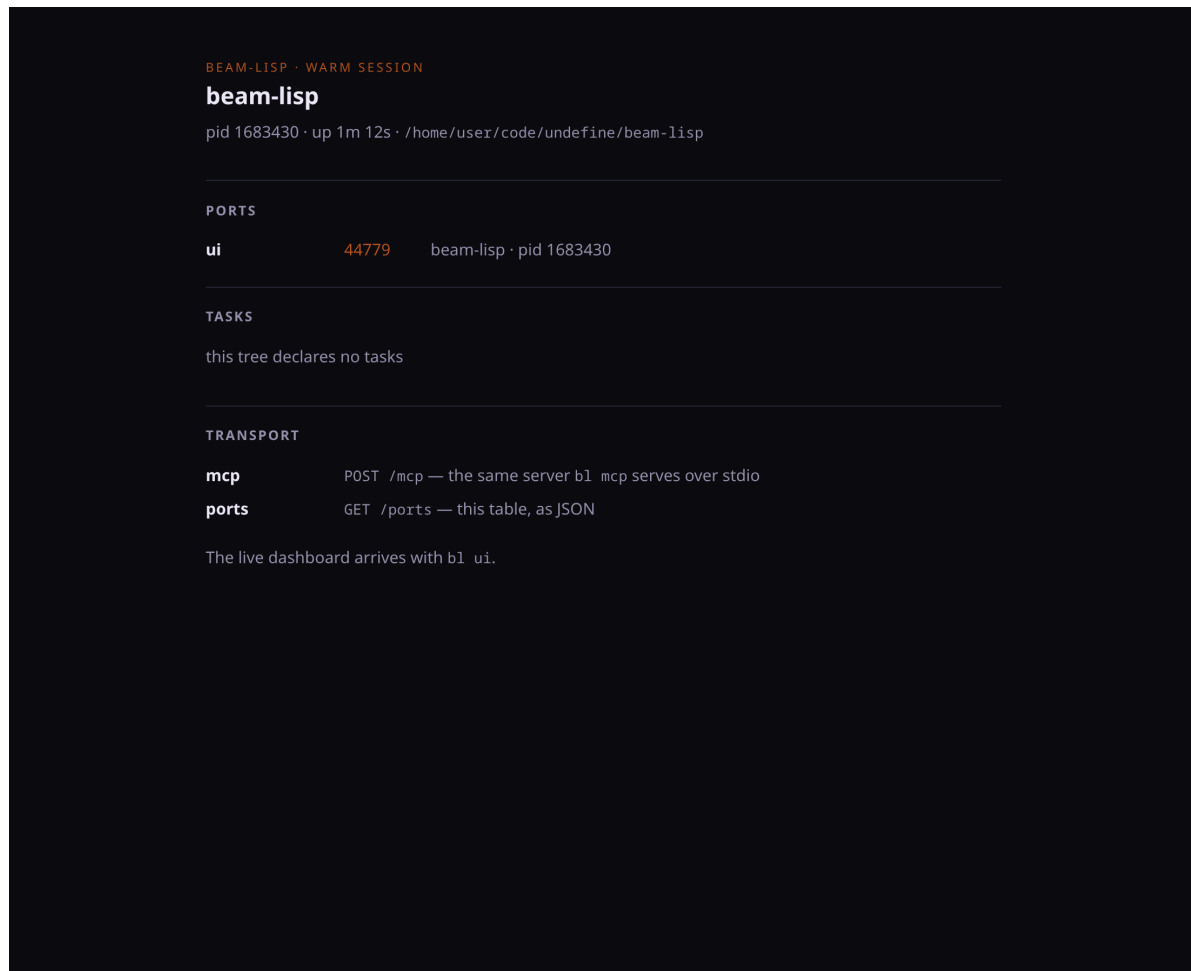


Figure 1: The session page, as served by a live daemon on this checkout: the tree, its age, its ports, its tasks, and how to reach MCP.

✓ The page, the port table and the MCP endpoint are one server

`v(0.35em) block(width: 100%, inset: (left: 1.1em))`

```
$ curl -s http://127.0.0.1:44779/ports
```

```
[{"claimed_at":1789215018,"name":"ui","pid":1683430,"port":44779,
  "root":"/home/user/code/undefine/beam-lisp","tree":"57461829a20cc2af"}]
```

```
$ bl ports
```

```
ui 44779 beam-lisp (pid 1683430)
```

the verb reads the same claims the page shows — the registry, not a copy of it

]

✓ MCP over HTTP is the same server as MCP over stdio

v(0.35em) block(width: 100%, inset: (left: 1.1em))[POST /mcp with tools/list returns the tools of mcp.server — the dispatch the stdio transport calls. Two transports, one server, identical capabilities.

```
$ POST /mcp (a tools/list request with the protocol version in meta)
```

```
{"result": {"tools": [{"name": "code/list", ...}, {"name": "code/query", ...}, ...]}}
```

The daemon-side test asserts exactly this (test/beam-lisp/daemon_ports_test.exs), because “the HTTP endpoint exists” is not the claim — “it is the same MCP” is.]

DECISION

The MCP index mounts on the FIRST /mcp request, not at boot.

Because — Mounting the codebase takes seconds, and a session nobody asks for MCP should not pay for it — the same rule every lazy verb follows. The cost lands on the request that wants it, where it is visible, rather than in a warm-up thread nobody can see.

The bug this wave uncovered

The first /mcp request failed with no source for codebase on the load path, in a namespace that ships in priv/std. The cause was in the loader, and it was not MCP-specific: a namespace whose ns form carries reader metadata declares itself as

```
(ns ^{:instr {:for "mcp" ...}} codebase ...)
```

and declared_ns/1 — the function that checks a candidate file really declares the namespace being loaded — read the token after ns, which is ^{...} rather than the name. So find_file/1 rejected the file as {:wrong_ns, ...} and source_content("codebase") answered nil for a file sitting in the search path. examples/mcp-demo.bl had been failing this way already.

DECISION

declared_ns/1 now skips reader metadata before the name — repeated, because metadata stacks (^:private ^:const x), and with nesting and strings honoured (^{:doc "a } in a string"} is not a delimiter).

Because — The declared name is the first SYMBOL after ns. Metadata is not a symbol, so reading it as the name was never a correct parse — it merely went unnoticed until a namespace that ships in priv used the form.

✓ The fix, on the example that had been failing

v(0.35em) block(width: 100%, inset: (left: 1.1em))

```
$ hl run examples/mcp-demo.bl
```

```
✓ All 11 exchanges passed (2026-07-28 envelopes, real self-index)
```

11 MCP exchanges — discover, tools, resources, prompts, subscribe/poll, a bad version — end to end

]

Numbers, and how they were checked

NEW MODULE

lib/beam_lisp/daemon/ports.ex — claim · release · list · port_of · holder

NEW MODULE

lib/beam_lisp/daemon/http.ex — the page · /ports · /mcp

TESTS

9 ExUnit (test/beam_lisp/daemon_ports_test.exs) + 6 bl (test/bl/system/ports_test.bl)

ephemeral assignment · idempotence · cross-session refusal · stale sweep · busy port · the page · the MCP dispatch · the message's three shapes

DOGFODDED

The screenshot above is a real daemon for this checkout, opened in a real browser, and the transcripts are that session's own output. The port claim in the screenshot (`ui 44779`) is the claim `bl ports` printed and `/ports` served — one registry, three readers.

W2 · W4 Tasks, and one runner instead of two

SHIPPED

The project declares its verbs

`env.bl` names the things a developer types. `bl tasks` lists them; `bl dev` runs one; the dashboard offers them as buttons. Dispatch is a FALLTHROUGH — builtins first, the project's tasks after — so `bl test` keeps meaning `bl test` whatever a file declares. A task that DOES collide is dropped and reported rather than silently losing:

```
$ bl tasks (this repository's own env.bl)
```

```
demo    - the Pulse walkthrough: one app, backend and frontend
          /home/user/code/undefine/beam-lisp/examples/live/11-pulse-app.bl
runner  - the isolated test runner, running four adversarial files
          /home/user/code/undefine/beam-lisp/examples/reload/10-ward-warm-runner.bl
```

DECISION

A task is a saved INVOCATION, not a second execution model: it runs its file the way `bl run FILE` does, through the CLI's own path — the daemon's intent endpoint calls exactly that, so there is no second implementation of “run a task”.

Because — Every extra implementation is a place for the two to disagree, and a task that behaves differently warm and cold is a bug nobody would think to look for.

ward's interface, without ward

`bl ward` is gone. Its semantics are what `bl test` IS: each file in its own env fork off one warm image, coherence-checked, contained — a file that explodes at load cannot take the run down, and one file's world is invisible to the next.

```
$ bl test test/bl/system/tasks_test.bl
```

— ward: warm isolated test run —

✓ system.tasks-test 8 passed

1 file(s) passed, 0 failed, 0 incoherent

✓ all green

DECISION

`bl ward` answers with ONE line pointing at `bl test` and exits 2.

Because — the migration note costs a line and the alias costs a permanent second word.

```
$ bl ward x.bl
```

bl: ward is now `bl test` — every file in its own isolated fork is the default (--shared opts out)

Instead of — an alias, which would keep two names alive for one thing — the shape this work exists to remove,

LAW · ONE RUNNER, TWO ESCAPES

`--shared` runs every file in one image (the old behavior, for a suite that genuinely needs a shared world) and `--async` selects the concurrent runner. Both are documented where the command is; neither is the default, because neither is what a test wants.

✓ The retired verb left the command table

`v(0.35em) block(width: 100%, inset: (left: 1.1em))` [`test/bl/system/test_verb_test.bl` asserts `commands` has `test` and has no `ward`, and that the help text names `--shared`. The runner's own shapes are exercised in `test/beam_lisp/test_verb_test.exs`, seven cases: a green suite, a failing one with the why, a load crash contained beside a passing sibling, two files that cannot see each other's world, `--shared`, `--json`, and the same suite served warm through the daemon.

That split is not tidiness: the isolated runner is not reentrant on itself. A bl-language test file may not start a suite inside a suite — the first version of this test did, and hung until the timeout — so the runner's own tests live where no outer ward is running.]

W5 · W6 One read-model, and the dashboard that renders it

SHIPPED

`bl daemon status` and the session page are two RENDERINGS of one value built by `BeamLisp.Daemon.Inspect`:

```
$ bl daemon status
```

```
bl daemon
tree      beam-lisp (/home/user/code/undefine/beam-lisp)
pid       2252356
tree_id   57461829a20cc2af
uptime_ms 33000
port      ui = 35969 (beam-lisp, pid 2252356)
ui        http://127.0.0.1:35969/
```

```
mcp      http://127.0.0.1:35969/mcp (the same MCP `bl mcp` serves over stdio)
tasks    demo · runner
queue_depth 0
```

The model answers, in one value: identity (tree, pid, age, compiler key), ports (every live claim), tasks (what the project declares), queue (is the session busy), and — when the language side has it loaded — the reload image.

DECISION

The model lives in Elixir (`BeamLisp.Daemon.Inspect`), not in a `bl.daemon.inspect` namespace as the plan sketched.

Because — its sources are Elixir-side (ports, the worker queue, the listeners); the one language-side source, `reload/inspect`, is reached through the runtime. A `bl` namespace would have had to reach the other way, through four bridges, to say less.

✓ A test compares the two faces rather than trusting them

`v(0.35em) block(width: 100%, inset: (left: 1.1em))[ test/beam_lisp/daemon_inspect_test.exs`
builds the model once and asserts both renderings against it: every live port appears in the text AND in the JSON with the same number, the session URL and its `/mcp` line reach both, and the JSON is JSON-safe (string keys, no structs). A field that reached one face and not the other fails there rather than in a browser.]

The dashboard

BEAM-LISP · WARM SESSION

beam-lisp

pid 2208116 · up 32s · /home/user/code/undefine/beam-lisp · [model.json](#)

TASKS

demo	the Pulse walkthrough: one app, backend and frontend /home/user/code/undefine/beam-lisp/examples/live/11-pulse-app.bl	run
runner	the isolated test runner, running four adversarial files /home/user/code/undefine/beam-lisp/examples/reload/10-ward-warm-runner.bl	run

PORTS

ui	36075	http://127.0.0.1:36075/ beam-lisp · pid 2208116
----	-------	--

IMAGE

the reload image is not loaded in this session — run `bl watch` or `bl monitor` to give it one

WORKER

queue_depth	0
compiler_key	3ac9ac2ebd0b84e0...

TRANSPORT

mcp	POST /mcp — the same server bl mcp serves over stdio
model	GET /model — the read-model this page is rendered from
terminal	bl daemon status — the same model, the terminal's face of it

Figure 2: The session page for this checkout: tasks with run buttons, the port table, the reload image (not loaded here — and the page says so), the worker, and the transports.

Clicking `run` is an INTENT: `POST /intent` names a task, the daemon runs it on its single worker — the same FIFO every client's command takes, so an intent never races a reload or a run — and the output comes back into the page.

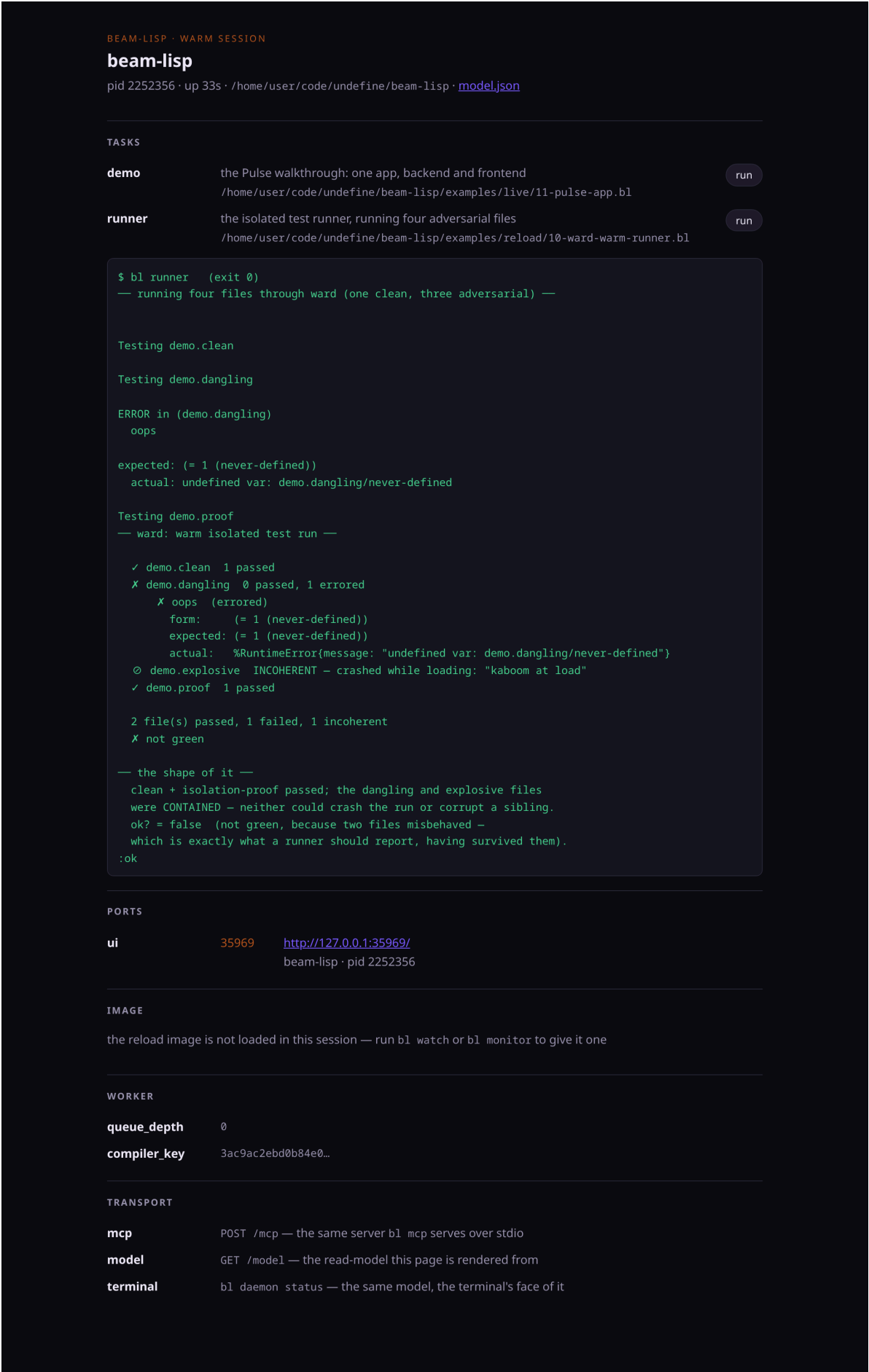


Figure 3: The `runner` task, run from the page: the isolated runner’s own report, exit 0, inline. The failure it reports is the `print_str` fix from the first wave — an error value printed as a value instead of crashing the printer.

DECISION

The write path requires the daemon's own token in `x-bl-token`, and the page carries it (server-rendered).

Because — the listener is loopback-only, and loopback is not a trust boundary: any page a developer visits can `POST` to `127.0.0.1`, and running a project's tasks on someone else's say-so is exactly the hole this face must not open. A cross-origin caller cannot read the token (no CORS) and cannot send a custom header without a preflight this server never approves. `test/beam_lisp/daemon_ports_test.exs` asserts the refusal.

if id != none or title != none { grid(columns: (auto, 1fr), column-gutter: 0.6em, align: (left + horizon, left + horizon), box(fill: ink-soft, inset: (x: 5pt, y: 2.5pt), radius: 1.5pt, text(7.6pt, fill: white, weight: 600, tracking: 0.5pt, font: sans, id)), text(10.4pt, weight: 600, font: sans, title),) v(0.5em, weak: true) } The plan promised a pulse-architecture client: a datom read-model, a pure view over it, minimal keyed patches, several viewers converging through the log. What shipped is the honest 80%: one read-model, server-rendered, with intents on the worker and a token guard. What is missing is the CLIENT half — no socket, no keyed diffs, no convergence between tabs.

The read-model is the part that had to be right, and it is: `GET /model` is the same value the page renders, so a live client is an addition rather than a rewrite. The dashboard REPL pane (FUP-038) waits on the same socket.

DOGFOODED

Every screenshot in this report is a daemon for THIS checkout, and the tasks in it are this repository's own `env.bl`. The `runner` task's output in the second shot is the ward example exercising the isolated runner — the same tool the test suites in these waves were verified with.

W7 Walkthroughs that run, and a repository that uses its own file

SHIPPED

The docs are programs

`docs/dev/env.bl.md` and `docs/dev/the-warm-session.bl.md` are livebooks: prose, then cells that RUN, with the result stored beside each one. `bl doc run --check` fails a build whose stored results no longer match its code, so a walkthrough cannot quietly go stale:

```
$ bl doc run docs/dev/env.bl.md --check
```

```
bl doc run: docs/dev/env.bl.md - 7 cells, 0 cell errors (checked, unchanged)
```

Every cell is deterministic by construction — temp dirs, no daemons, no timestamps — which is what lets them be checked at all. That constraint found one real thing: `u/kw [:root \" /x\" :pid 1]` SILENTLY DROPS the second pair, because `kw` takes one pair per argument. The cell that documented the port refusal was quietly asserting nothing until the stored result exposed it (`{:ok, 45153}` where a refusal was expected).

if id != none or title != none { grid(columns: (auto, 1fr), column-gutter: 0.6em, align: (left + horizon, left + horizon), box(fill: ink-soft, inset: (x: 5pt, y: 2.5pt), radius: 1.5pt, text(7.6pt, fill: white, weight: 600, tracking: 0.5pt, font: sans, id)), text(10.4pt, weight: 600, font: sans, title),) v(0.5em, weak: true) } (u/kw [:a 1 :b 2]) builds one pair and drops the rest — a caller that intended two gets no error and no second binding. The arity is discoverable (one vector per argument) but the failure is silent, and silent is the one thing a configuration builder must not be. A check on the vector's length, plus an error naming the offending form, would cost three lines.

The repository's own env.bl

The strongest dogfood available: this checkout now has an `env.bl`, and the tooling in it reads that file about itself.

```
{:name "beam-lisp"
 :paths ["examples" "priv/lib"]
 :tasks {:demo    {:run "examples/live/11-pulse-app.bl" ...}
         :runner  {:run "examples/reload/10-ward-warm-runner.bl" ...}}}
```

Two things fall out of it. `bl demo` and `bl runner` are now verbs this repository has — and the dashboard in the previous wave shows them as buttons. And the test corpus RESOLVES: `test/bl/` reaches into `examples/` for shapes, schemas and live apps, which is why `bl test test/bl/` used to fail on missing namespaces before it could fail on a failing test.

```
$ bl test test/bl/
```

```
161 file(s) passed, 8 failed, 1 incoherent
× not green
```

the corpus RUNS from the repo root now; the remaining failures are the corpus's own business, not the runner's

✓ Isolation holds across a 170-file corpus

v(0.35em) block(width: 100%, inset: (left: 1.1em))[Each of those files ran in its own env fork off one warm image. A file that crashed at load was contained and its siblings still ran — the property ward was built for, now just what `bl test` does.]

— What this cost, and what is left

CLOSING

Where the bugs came from

Every non-trivial bug this work uncovered was found by RUNNING something, not by reading it:

- A daemon request's fork could not reach the loader: a search path registered inside a request was invisible to the `Loader.Server`, so a project's declared roots never applied warm. (W1)

- `:os.getpid/0` answers a CHARLIST, so every port claim looked stale and was swept the moment it was written — the registry worked perfectly and was empty. (W3)
- The loader could not see a namespace whose `ns` form carries metadata, so `source_content(\"codebase\")` answered nil for a file sitting in the search path, and the MCP codebase mount had been failing — `examples/mcp-demo.bl` too. (W3)
- The isolated runner is not reentrant on itself: a bl-language test that starts a suite inside a suite hangs. Found by a 900-second timeout. (W4)
- Three separate places treated "is a map" as "is a beam-lisp map", in a codebase where eleven struct types are maps. The repository's own lint test caught all three at once. (W5/W6)

What is not done

- G1, above — the dashboard is server-rendered: one read-model, no keyed diffs, no convergence between tabs.
- G2, above — `u/kw` silently drops what it cannot express.
- The dashboard REPL pane (FUP-038) and the editing/file-browsing spike (FUP-039) remain follow-ups, as planned.
- One test in the full suite is red and NOT explained: a daemon-served `bl test` of a scratch suite fails in full-suite runs with `BeamLisp.Ns.Core."is-eq-report"/7 is undefined or private`, and passes when its own file is run alone (3 for 3). Filed as FUP-044 with the evidence and the three candidate causes. Recording it here rather than leaving a green suite to imply more than it showed.
- The plan listed five walkthroughs (`env.bl`, `daemon`, `tasks`, `ports`, `ui`); two shipped, covering the same ground in one pass each. The verb livebooks (`bl.env`, `bl.test`, `bl.ports`) are the other half of that documentation.

The shape of the thing

BEFORE

Twenty-one verbs, each finding a project's files its own way; a daemon and an isolated test runner that did not know about each other; warmth and isolation as commands a developer had to choose between.

AFTER

One project file the tooling reads about itself; one runner whose isolation is the default; one session per tree with one address; one read-model that the terminal and the browser both render. Warmth is an optimization, isolation is a property, and the CLI is a remote control.